

Fitness ML Tracker

End-to-End Project Report and Interview Preparation Guide

Repository: Shourya292005/Fitness-ML-Tracker

Prepared on: 2026-06-03

Purpose: This PDF explains the project from problem definition to final dashboard and model selection, with interview-ready answers for modeling, overfitting, accuracy, privacy, and future scope.

- Core project: a practical ML dashboard that processes health telemetry and forecasts next-week smoothed body weight.
- Strongest artifact: the Streamlit dashboard, now supported by screenshots and reproducible scripts.
- Modeling outcome: regularized XGBoost was evaluated, but the safer baseline predictor was selected because it generalized slightly better.
- Interview angle: the project shows data engineering, leakage control, baseline comparison, and honest model evaluation.

1. Executive Summary

Fitness ML Tracker is an end-to-end data science project built around real-world biometric telemetry. It takes noisy daily health data, cleans missing and irregular values, engineers rolling-window features, evaluates a next-week weight predictor, and presents insights in a Streamlit dashboard.

The project is valuable in interviews because it goes beyond a notebook demo. It includes data cleaning, feature engineering, model serialization, baseline-aware evaluation, screenshots, sample data, and a dashboard that a reviewer can understand quickly.

One-minute interview pitch: I built a health analytics dashboard that turns messy fitness-tracker and nutrition data into rolling calorie-balance features and next-week weight forecasts. I initially tested XGBoost, found overfitting on a small flat dataset, and corrected the final artifact by selecting the simpler baseline model that performed better on chronological holdout data.

- Problem solved: personal health logs are fragmented, noisy, and difficult to inspect over time.
- Technical focus: cleaning, temporal alignment, rolling features, model evaluation, and dashboarding.
- Responsible ML point: the report explains why the baseline is currently safer than the complex model.

2. Project Lifecycle: Start to End

The project can be explained as a complete workflow rather than isolated scripts. This is useful in interviews because it shows ownership from raw data through user-facing output.

- 1. Define the task: forecast near-term body-weight trend from calorie and activity signals.
- 2. Collect telemetry: combine body weight, calories, active energy, basal energy, steps, protein, carbs, and fat.
- 3. Clean raw data: sort by date, interpolate tracker signals, and impute nutrition gaps.
- 4. Engineer features: create TDEE, net calorie balance, rolling averages, and shifted targets.
- 5. Train and evaluate: test regularized XGBoost against a rolling-weight baseline.
- 6. Diagnose fit: compare train and holdout metrics to catch overfitting.
- 7. Deploy-style artifact: save the selected predictor and load it inside Streamlit.

Raw input	data/raw/health_wide_full.csv
Feature pipeline	scripts/build_features.py and data/raw/cleaning_data.py
Model training	model/train_model.py
Dashboard	src/app.py

3. Data Understanding and Privacy

The original dataset contains personal biometric information, so the repository includes a small anonymized sample instead of exposing the full private source. This is an important interview point because it balances reproducibility with privacy.

- Main columns: Date, Body Weight, Calories, Protein, Active Energy, Basal Energy, Steps, Carbs, and Fat.
- Data challenge: tracker readings can be missing, nutrition logs can be incomplete, and sampling can be irregular.
- Privacy step: sample data shifts dates and scales values while preserving the schema.
- Verification role: the sample is a smoke test for pipeline behavior, not the final validation set.

Interview answer: The raw data is personal, so I anonymized a small sample to make the repository runnable and reviewable without leaking private health information.

4. Data Cleaning Pipeline

The cleaning layer prepares raw daily telemetry for feature engineering and model training. The important idea is not simply removing missing values; it is preserving the chronological meaning of each day.

- Date parsing and sorting: records are converted to datetime and ordered chronologically.
- Wearable interpolation: Basal Energy and Active Energy are linearly interpolated because they are continuous daily signals.
- Nutrition imputation: missing Calories and Protein are filled so the feature pipeline remains complete.
- Signal construction: Tracker_TDEE is built from Basal Energy plus Active Energy.
- Output writing: cleaned and engineered data are saved into processed and model-ready paths.

Interview answer: I used interpolation for continuous wearable metrics because neighboring days often provide a reasonable estimate. For nutrition logs, imputation keeps the pipeline stable while still being named as a limitation.

5. Feature Engineering and Target Design

Feature engineering is the heart of the project. The model does not learn directly from raw noisy values; it learns from smoothed, time-aware signals.

- Tracker_TDEE: Basal Energy plus Active Energy, representing tracker-estimated daily expenditure.
- Net_Calorie_Balance: Calories_Imputed minus Tracker_TDEE, representing surplus or deficit.
- Weight_7Day_Avg: smoothed body weight to reduce daily water-weight noise.
- Net_Calorie_7Day_Avg: smoothed calorie balance to avoid overreacting to one-day logs.
- Target_Weight_Next_Week: future 7-day smoothed weight shifted back by seven rows.

Leakage control: The chronological split avoids shuffling because this is a time-series problem. Shuffling would let future-like records leak into training and make metrics unrealistic.

6. Dashboard and User Experience

The Streamlit dashboard is the most visible part of the project. It translates the pipeline into something a recruiter or user can understand without reading code.

Smart Calories Tracker - Dashboard

Interactive Trends

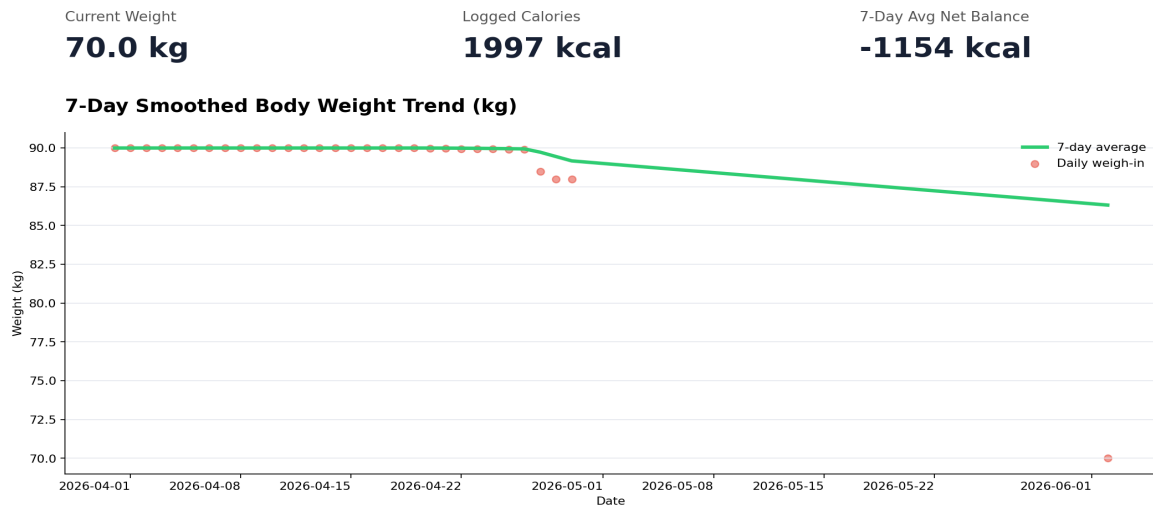


Figure 1. Interactive trends view with rolling body-weight behavior.

- Sidebar inputs support daily calories, macros, body weight, and optional energy metrics.
- KPI cards summarize current weight, logged calories, and 7-day net calorie balance.

7. AI Prediction and Historical Ledger

Smart Calories Tracker - Dashboard

AI Predictions

Current Weight

70.0 kg

Logged Calories

1997 kcal

7-Day Avg Net Balance

-1154 kcal

Next Week Forecast

Predicted Target Weight (In 7 Days)

86.32 kg

Expected weekly change: +16.32 kg



Figure 2. Prediction view backed by the saved model artifact.

The AI tab loads the serialized predictor and creates a next-week forecast from the latest rolling features.

Smart Calories Tracker - Dashboard

Historical Data Ledger

Current Weight

70.0 kg

Logged Calories

1997 kcal

7-Day Avg Net Balance

-1154 kcal

Date	Weight	Calories	Protein	Active Energy	Net Balance
2026-06-02	70.0	1997.0	170.0	999.2	130.6
2026-04-29	88.0			999.2	-56.4
2026-04-28	88.0	1681.0	86.2	804.7	-1317.1
2026-04-27	88.5	2671.5	243.2	1821.6	-1458.3
2026-04-26	89.9	1802.3	161.7	1168.3	-1600.6
2026-04-25	89.9			188.8	-471.0
2026-04-24	89.9	678.3	49.4	1761.2	-3305.7
2026-04-23	89.9	1825.2	158.4	1144.4	-1516.7
2026-04-22	90.0	1565.5	138.1	1105.2	-1740.1
2026-04-21	90.0			301.8	-525.9
2026-04-20	90.0	1810.0	146.5	1602.4	-1940.4
2026-04-19	90.0	1686.0	156.8	1273.5	-1822.7
2026-04-18	90.0			162.8	-338.1
2026-04-17	90.0			333.6	-525.7
2026-04-16	90.0			397.2	-682.7

Figure 3. Historical ledger for row-level inspection.

8. Modeling Strategy: Why This Model?

XGBoost was a reasonable first model because the dataset contains structured tabular signals and potentially nonlinear relationships between calorie balance, activity, and weight trend.

- Why XGBoost was tried: it handles nonlinear tabular signals and is common for structured ML problems.
- Why not only linear regression: energy balance and weight change may have nonlinear effects and lagged behavior.
- Why not deep learning: the dataset is too small; a neural model would overfit faster and be harder to justify.
- Why a baseline was required: a current-weight forecast is a strong benchmark in short-horizon weight prediction.

Important interview answer: I did not choose the complex model blindly. I trained a regularized XGBoost model, compared it against a simple baseline, and selected the baseline when it generalized better.

9. Evaluation Results and Accuracy

Because this is a regression problem, accuracy is expressed through error metrics and tolerance bands rather than classification accuracy. The most interview-friendly summary is that the selected model predicts within ± 1.0 kg on 94.1% of holdout rows.

Model	MAE	RMSE	R2	± 0.5 kg	± 1.0 kg
Regularized XGBoost	0.3236 kg	0.9271 kg	-0.1387	82.4%	94.1%
Selected baseline	0.3212 kg	0.9214 kg	-0.1249	82.4%	94.1%

- MAE: average absolute prediction error in kilograms.
- RMSE: penalizes larger errors more strongly, useful for outlier sensitivity.
- R2: negative here because the dataset is small and the holdout period has distribution shift.
- Tolerance-band accuracy: within ± 0.5 kg and ± 1.0 kg is easier to explain to non-technical reviewers.

Accuracy statement: Selected model holdout performance is MAE 0.3212 kg, RMSE 0.9214 kg, within ± 0.5 kg on 82.4% of rows, and within ± 1.0 kg on 94.1% of rows.

10. Overfitting Diagnosis and Correction

The first XGBoost version showed a classic overfitting signal: it achieved 0.0000 kg train MAE while holdout MAE stayed around 0.3236 kg. That means the model memorized the flat training period instead of learning a robust relationship.

- Observed the gap: training error was perfect but test error was not.
- Regularized XGBoost: reduced estimators, learning rate, tree depth, and added alpha/lambda regularization.
- Compared against baseline: the rolling-weight baseline still had slightly lower holdout MAE.
- Changed selection logic: `train_model.py` now saves the model with better chronological holdout MAE.
- Final result: the saved model is the baseline predictor, reducing overfitting risk for current data.

Interview answer: I corrected overfitting by adding a baseline-aware selection rule. Instead of deploying the complex model because it sounded impressive, I selected the simpler model that performed better on unseen chronological data.

11. Reproducibility and Runbook

A reviewer can reproduce the core workflow from the repository root. This matters because it shows the project is not only a static showcase.

Install dependencies	<code>pip install -r requirements.txt</code>
Launch dashboard	<code>streamlit run src/app.py</code>
Build features	<code>python scripts/build_features.py</code>
Train and evaluate	<code>python model/train_model.py</code>
Evaluate sample data	<code>python scripts/evaluate_sample_data.py</code>
Regenerate screenshots	<code>python scripts/export_dashboard_screenshots.py</code>

- The model is serialized with Joblib so the dashboard can load it without retraining.
- The README includes screenshots so recruiters can see the dashboard before running it.
- The sample dataset makes the repository easier to inspect while preserving privacy.

12. Interview Questions and Strong Answers

Why this project?	It shows an end-to-end workflow: noisy data, feature engineering, model evaluation, dashboarding, and documentation.
Why XGBoost?	It is a strong tabular model for nonlinear relationships, but I verified it against a simpler baseline.
Why not deploy XGBoost?	It overfit the small flat training period and did not beat the baseline on chronological holdout MAE.
How reduce overfitting?	I regularized XGBoost, measured train-vs-test error, compared against baseline, and saved the better generalizing predictor.
What is the accuracy?	On holdout data, the selected model has 0.3212 kg MAE and is within +/-1 kg for 94.1% of rows.
What improve next?	Collect more varied data, add lag features, use rolling-origin validation, and evaluate robust models.
Short recruiter answer: I built an end-to-end fitness telemetry ML tracker with Streamlit, Pandas, XGBoost evaluation, baseline-aware model selection, and Joblib deployment.	

13. Limitations and Future Improvements

The main limitation is not the algorithm; it is the data. The training period is small and relatively flat, which makes it easy for many models to look perfect on train data and weak on future data.

- More data: collect longer history with more varied weight movement and dietary phases.
- Better validation: use rolling-origin validation instead of a single split when enough data exists.
- More features: add lagged weight change, adherence streaks, weekly variance, steps averages, and macro balance.
- Model monitoring: track prediction error over time and retrain only when new data improves validation.
- Dashboard polish: add confidence messaging and exportable summaries.

Closing answer: The project is strongest because it is honest. It does not hide that XGBoost overfit. It shows that I can diagnose a model problem, choose a simpler production-safe option, and explain the tradeoff clearly.

14. Final Talking Points

- This is not just an ML script; it is a full product-style analytics workflow.
- The dashboard is the strongest visual proof and is documented with screenshots.
- The pipeline demonstrates cleaning, imputation, rolling-window features, target shifting, and model serialization.
- The model card is honest: the selected baseline beats regularized XGBoost by a small margin on holdout MAE.
- The overfitting correction is the strongest interview story because it shows judgment, not just tool use.
- The next step is to collect more varied data before trying to justify a more flexible model again.

Best final interview answer: I built a complete fitness ML tracker, found that XGBoost overfit the small dataset, corrected it with baseline-aware model selection, and documented the dashboard, data, model metrics, and limitations clearly.